

Variation 2: Confidence-Aware Convex Heuristic Search

A Mathematical and Algorithmic Framework

Research Proposal Specification

Abstract

This report formalizes a hybrid search architecture that dynamically blends a learned, ranking-based neural heuristic with a classical admissible heuristic. By estimating epistemic uncertainty online via ensemble rank variance, the system leverages the extreme tie-breaking speed of neural networks on familiar distributions while enforcing a mathematically safe fall-back mechanism when encountering out-of-distribution structures.

1 Introduction

Classical heuristic search algorithms (e.g., A^*) rely on admissible heuristics, such as Manhattan distance, to guarantee optimal pathfinding. However, these heuristics often produce massive plateaus of identical scores, forcing the search to expand thousands of unnecessary nodes due to tied priorities.

Conversely, learned heuristics trained via pairwise ranking losses aggressively break these ties, yielding highly efficient search trajectories. Yet, pure neural heuristics are brittle; they hallucinate catastrophic paths when faced with out-of-distribution (OOD) states. The **Confidence-Aware Convex Heuristic Search** resolves this by treating the neural network as an “advisor” whose influence is dynamically gated by its internal confidence.

2 The Dual-Expert Formulation

The framework evaluates frontier states using two distinct heuristic sources:

1. **The Learned Ranker (H_{nn}):** An ensemble of M neural networks trained via L^* ranking loss. For a given state s , seed m outputs an arbitrary scalar $h_m(s)$. This expert is scale-invariant and designed exclusively to rank states relative to one another.
2. **The Classical Admissible Heuristic (H_{class}):** A geometric heuristic (e.g., Manhattan distance) outputting true grid steps $h_{class}(s)$. It is robust and guarantees goal convergence, but suffers from deadlock blindness and slow tie-breaking.

3 Measuring Epistemic Uncertainty

Because the outputs of H_{nn} are uncalibrated scalars, computing standard numerical variance is invalid. Uncertainty must be measured through *Rank Variance*.

3.1 Percentile Rank Conversion

When a state s is evaluated alongside a batch of N frontier states, each seed m independently sorts the batch. We compute the normalized percentile rank $p_m(s) \in [0, 1]$:

$$p_m(s) = \frac{\text{Rank of } s \text{ under seed } m - 1}{N - 1} \quad (1)$$

where a rank of 1 (best) maps to 0.0, and the worst rank maps to 1.0.

3.2 Rank Variance Calculation

The mean percentile $\bar{p}(s)$ and the rank variance $\sigma_{rank}^2(s)$ across the M seeds are calculated as:

$$\bar{p}(s) = \frac{1}{M} \sum_{m=1}^M p_m(s), \quad \sigma_{rank}^2(s) = \frac{1}{M} \sum_{m=1}^M (p_m(s) - \bar{p}(s))^2 \quad (2)$$

The theoretical maximum of $\sigma_{rank}^2(s)$ is 0.25 (complete polarization of the ensemble).

3.3 The Confidence Gate (λ)

We convert rank variance into a confidence multiplier $\lambda(s) \in [0, 1]$. To prevent the system from completely discarding the network's structural priors (e.g., avoiding dead ends), we enforce a safety floor λ_{min} (e.g., 0.25):

$$\lambda_{raw}(s) = \max(0, 1 - 4 \cdot \sigma_{rank}^2(s)) \quad (3)$$

$$\lambda(s) = \lambda_{min} + (1 - \lambda_{min}) \cdot \lambda_{raw}(s) \quad (4)$$

4 The Scale-Matched Convex Blend

To blend a percentile $p \in [0, 1]$ with an actual step distance, we must dynamically project the neural network's rank onto the scale of the classical heuristic.

Let μ_{min} and μ_{max} represent the running historical minimum and maximum values produced by H_{class} during the search. We scale the mean neural percentile $\bar{p}(s)$ into this domain:

$$\hat{h}_{nn}(s) = \mu_{min} + \bar{p}(s) \cdot (\mu_{max} - \mu_{min}) \quad (5)$$

Both heuristics now operate on comparable scales. We blend them using the confidence gate:

$$h_{blend}(s) = \lambda(s) \cdot \hat{h}_{nn}(s) + (1 - \lambda(s)) \cdot h_{class}(s) \quad (6)$$

The final evaluation function for the A^* priority queue is:

$$f(s) = g(s) + h_{blend}(s) \quad (7)$$

where $g(s)$ is the true accumulated path cost.

5 Algorithmic Implementation and Optimizations

Implementing this mathematical framework requires specific engineering optimizations to maintain the $O(\log N)$ efficiency of the A^* priority queue and maximize GPU throughput.

5.1 Execution Workflow

- **Step 1: Batched Expansion.** Pop a batch of B top nodes from the Open list. Generate all valid child transitions to form a candidate pool of size K .
- **Step 2: Classical Evaluation.** Calculate the accumulated cost $g(s)$ and classical heuristic $h_{class}(s)$ for all K candidates. Update running statistics μ_{min} and μ_{max} .
- **Step 3: Ensemble Inference.** Pass the K candidates through the M neural network seeds in a single batched tensor operation to extract arbitrary ranking scores.
- **Step 4: Rank & Blend.** Compute percentile ranks $p_m(s)$ within the batch K . Calculate rank variance $\sigma_{rank}^2(s)$, project to $\hat{h}_{nn}(s)$, and apply the convex blend to yield $h_{blend}(s)$.
- **Step 5: Queue Insertion.** Push the evaluated candidates into the global Min-Heap using their final $f(s)$ scores.

5.2 Engineering Fixes for Deployment

1. **Static Queue Priorities:** $f(s)$ values must remain static once pushed to the Min-Heap. Percentiles and projections must be calculated against a *running historical window* (e.g., the current expansion batch or EMA stats) rather than the entire global Open list, preserving $O(\log N)$ queue operations.
2. **Bounded Sub-optimality:** By adopting a learned heuristic (even dynamically), the search relinquishes strict admissibility. The system transitions from an optimal A^* to a Bounded Sub-optimal A^* , trading exact shortest-path guarantees for massive reductions in node expansions.